Introduction to Stochastic Optimisation

Reference

Essentials of Metaheuristics, 2nd Ed.
Sean Luke, Lulu, 2016

© Siwen Luo, UWA

The Real World is Messy

Recap

- Ability to compute 1st and/or 2nd derivative is a BIG assumption
- Relatively *small* class of "well-behaved" functions
- In many cases we can't compute the derivative because *we don't even know what the function is!*
- Remember that "black box"…

Black Box Optimisation

Candidate solution (hypothesis) could be:

- a mathematical function (eg a neural network)
- a program that converts text to speech for Siri
- a program for a walking robot →
- a classifier that tells you when to buy Bitcoin
- a job shop schedule that maximises output of a BYD factory…

Human Robots Rise. Now They Can Walk?
IEEE Spectrum, 2010

Basic Requirements (review)

- A *description (modelling) language*
  - mathematical
  - computer program
  - tree, graph, network
  - rules (logic, grammar, planner,…)
- A way of expressing a *candidate solution* — may be *executable*
- A way of *assessing* how well the candidate solution performed

Optimisation Algorithm - Very Basic Operations

1. Generate a candidate solution (from the hypothesis space)
2. Determine the quality of the candidate solution
3. Repeat until good enough!

- ***Trial and error*** approach
- Note the importance of the choice of hypothesis space (and language) even for this simplest of algorithms!
  - ➡ If a good solution doesn't exist in the space, you're never going to find one!

Basic Operations and Heuristics

1. Generate a candidate solution (from the hypothesis space)
2. Determine the quality of the candidate solution
3. Repeat until good enough!

- Clearly this algorithm may take a *long* time, maybe forever
- To improve performance, we want to use some kind of ***heuristics***
  - "rules of thumb", things that seem to make sense in many situations,..
- One overarching heuristic might be:

  "its often easier to find a good solution by modifying one you've found already than by starting from scratch each time"

  - concept of *incremental improvement…*
  - initial guess might be bad, but at least we can see if we're making progress!

The "Tweak"

its often easier to find a good solution by modifying one you've found already than by starting from scratch each time

Luke (2016) refers to this as '*tweaking*' a current candidate solution, so we will use this terminology too

- Looking carefully, there are some more assumptions built in to Luke's "tweak". Luke states:

  "Sometimes it finds worse ones nearby, sometimes it finds better ones."

  - *"nearby"* (and tweak) suggest "small" changes (whatever that means)
  - *presumption* that better solutions are within the "*vicinity*" of good solutions found so far (whatever that means)

- Luke's heuristic is probably more like:

  its often easier to find a good solution by modifying ***a good-ish*** *one you've found already* ***by a small amount*** *than by starting from scratch each time*

Basic Operations - Modification

1. Provide one or more initial candidate solutions (*initialisation*)
2. Determine the quality of candidate solutions (*assessment*)
3. *Modify* a candidate solution
   - make a copy
   - "*tweak*" the copy to produce a new candidate

   …

- Note - steps in gradient methods, pattern search, Simplex, etc can be considered examples of (deterministic) tweaks
- Here there is no assumption of determinism - for example, may be deterministic, probabilistic/random, random choice,…

Basic Operations - Selection

- Our incremental improvement heuristic

  "it's often easier to find a good solution by modifying (a good-ish) one you've found already (by a smallish amount) than by starting from scratch each time

  needs a couple more things:

  - a way to store solution(s) found so far and their quality
  - a way to select a candidate to modify (*selection*)

- We now have (almost) enough for a simple gradient-ascent-like algorithm that doesn't use the gradient…

Hill Climbing

Algorithm 4 *Hill-Climbing*

```
S ← some initial candidate solution                    ▷ The Initialization Procedure
repeat
    R ← Tweak(Copy(S))                                 ▷ The Modification Procedure
    if Quality(R) > Quality(S) then                    ▷ The Assessment and Selection Procedures
        S ← R
until S is the ideal solution or we have run out of time
return S
```

- We haven't specified *how* to tweak - this will depend on the hypothesis language
- eg. add a number, change a weight, swap lines of code, prune a branch of a tree, change a grammar rule,...
- Usually it will involve some degree of ***randomness***.

Steepest Ascent Hill Climbing

"We now have (almost) enough for a simple gradient-ascent-like algorithm that doesn't use the gradient..."

- Why "almost"?

- In gradient ascent we don't just pick *any* uphill direction - we use the grad ∇ to pick *steepest*
- Hill Climbing algorithm is *opportunistic* - perhaps a little like a randomised version of opportunistic generalised pattern search
- We could get a better guesstimate of the steepest direction by more ***sampling***
  - More like a randomised version of Hooke-Jeeves
- How much sampling?
  - We've seen similar trade-offs before, eg. sample size in pattern searches

Steepest Ascent Hill Climbing

Algorithm 5 *Steepest Ascent Hill-Climbing*

```
1:  n ← number of tweaks desired to sample the gradient

2:  S ← some initial candidate solution
3:  repeat
4:      R ← Tweak(Copy(S))
5:      for n − 1 times do
6:          W ← Tweak(Copy(S))
7:          if Quality(W) > Quality(R) then
8:              R ← W
9:      if Quality(R) > Quality(S) then
10:         S ← R
11: until S is the ideal solution or we have run out of time
12: return S
```

Another variation

- instead of comparing R and S (9-10) just replace
- but store the best solution achieved so far…

Steepest Ascent Hill Climbing with Replacement

Algorithm 6 *Steepest Ascent Hill-Climbing With Replacement*

```
n ← number of tweaks desired to sample the gradient

S ← some initial candidate solution
Best ← S
repeat
    R ← Tweak(Copy(S))
    for n − 1 times do
        W ← Tweak(Copy(S))
        if Quality(W) > Quality(R) then
            R ← W
    S ← R
    if Quality(S) > Quality(Best) then
        Best ← S
until Best is the ideal solution or we have run out of time
return Best
```

- note S can get lost, hence use of *Best*
- introduces the idea of an "***elite group***" (called *Best* here) which we'll see again
- not restricted to uphill?

Some Nomenclature

Notice in hill climbing (Alg 4) we pick (select) a winner in each iteration from 1 existing candidate (S) and 1 modified candidate (R)

- we could call this a **(1+1)** algorithm

Steepest ascent hill climb (Alg 5)

- select from 1 existing and n modified
- call this **(1+*n*)**

Steepest ascent HC with replacement (Alg 6)

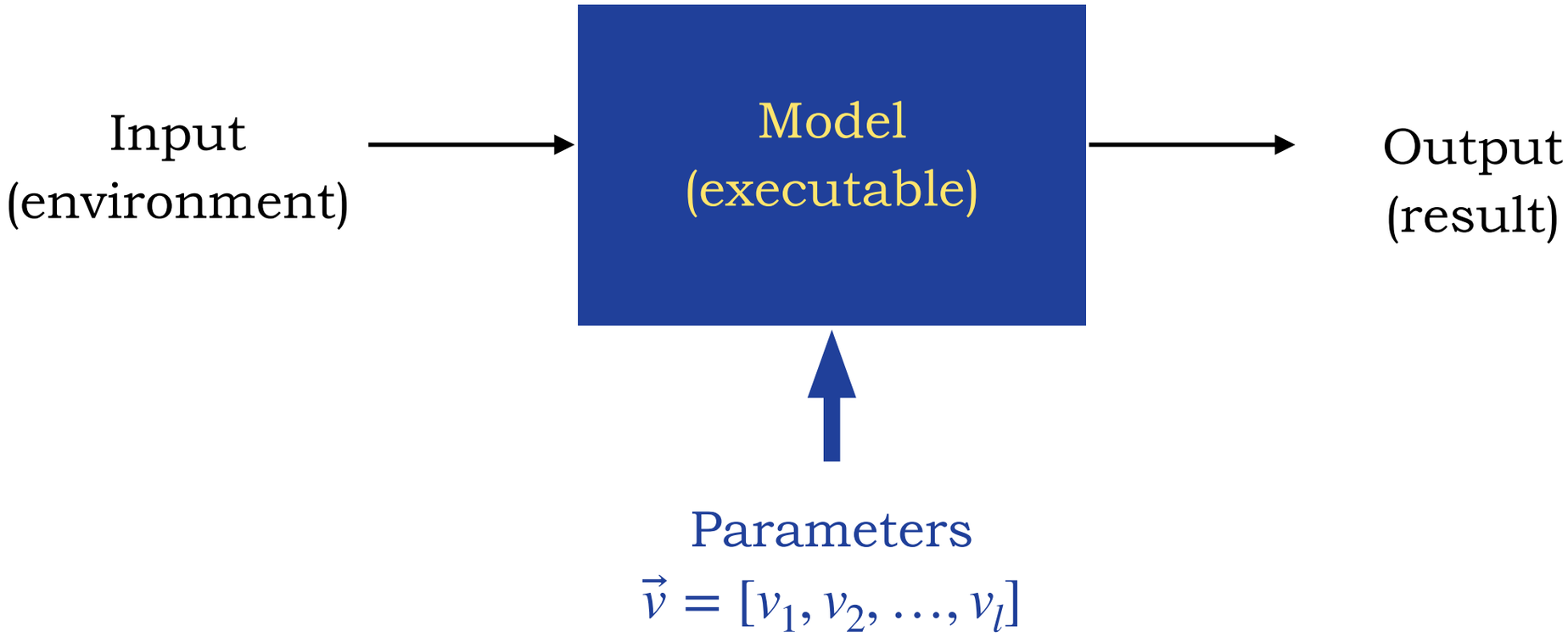
- select from n modified copies
- could call this **(1, *n*)**

We'll come back to this later.

Tweaking (Again)

- Could involve many things
  - switch branches of a tree
  - add a new node to a neural net
  - change the order of machines in a job-shop schedule
  - etc…
- In *many* situations however we are able to choose a representation where the tweaking can be done by changing real numbers

(Real value) Parameterised Models



Candidate solution is *defined* by a vector of parameters.

- Numerical parameters are nice to work with
  - modifications are mathematical operations
  - tweaks can typically use probability distributions

Hyper-Parameters

Merriam-Webster

hyper- prefix

Definition of *hyper-* (Entry 2 of 2)

1 : above : beyond : SUPER-

*// hyper*market

- Parameter works “inside” the model
- *Hyper-parameter* works “above” or “outside” the model
  - eg. determine the operation of the algorithm

- We have seen hyper-parameters before - eg. step size α, tolerance ϵ
  - Algs 5 and 6 have a new one n - determines number of comparisons/ evaluations

Hyper-Parameters

Hyper-parameters often make very important trade-offs in how, how fast, and even whether an optimisation algorithm works.

Can often be imagined as “tuning dials” for the algorithm.

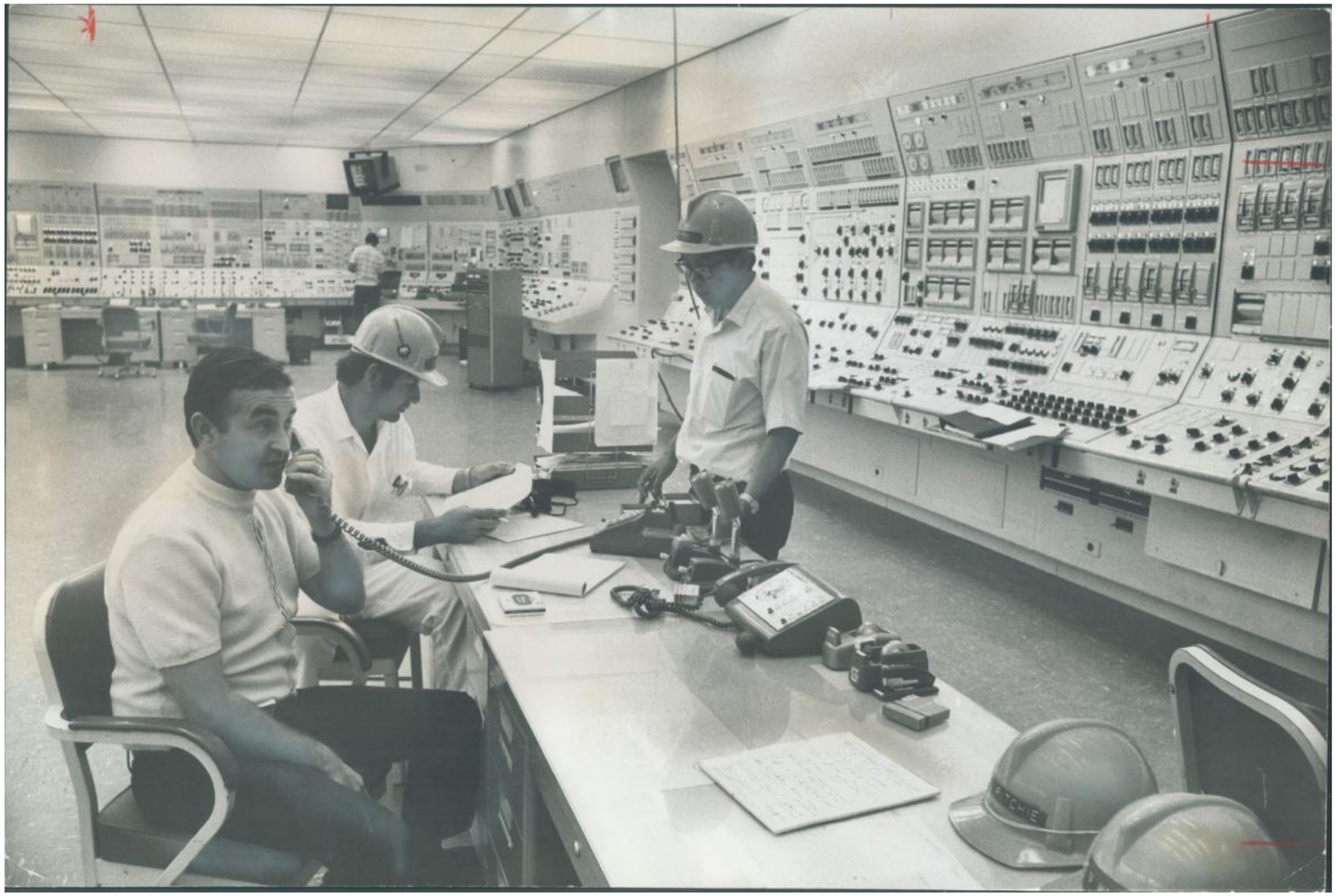


Image: Toronto Public Library

Hyper-Parameters

In this case the hyper-parameter n determines a ***computational trade-off***

- (cf. CCS vs Hooke-Jeeves)

Discussion:

- What trade-off is n making?

  Hint: Think about a fixed computational resource.

- Can you think of examples of domains (landscapes) where different choices of n (small or large) might be more successful?

  (Assume a 2-d solution space.)

Initialisation

- Luke assumes bounds - *min* and *max* - and *uniform* distribution:

Algorithm 7 *Generate a Random Real-Valued Vector*

```
min ← minimum desired vector element value
max ← maximum desired vector element value

v⃗ ← a new vector ⟨v_1, v_2, ...v_l⟩
for i from 1 to l do
    v_i ← random number chosen uniformly between min and max inclusive
return v⃗
```

- More generally, *min* and *max* could be infinite.
- Q: What would be an example of an unbounded probability distribution?

Modification/Tweak

- Following Luke, (small) random noise added element-wise
  - p - probability of adding noise to an element ("coin toss")
  - r - half-range of (bounded) uniform noise

Algorithm 8 *Bounded Uniform Convolution*

```
1: v⃗ ← vector ⟨v_1, v_2, ...v_l⟩ to be convolved
2: p ← probability of adding noise to an element in the vector        ▷ Often p = 1
3: r ← half-range of uniform noise
4: min ← minimum desired vector element value
5: max ← maximum desired vector element value

6: for i from 1 to l do
7:     if p ≥ random number chosen uniformly from 0.0 to 1.0 then
8:         repeat
9:             n ← random number chosen uniformly from −r to r inclusive
10:        until min ≤ v_i + n ≤ max
11:        v_i ← v_i + n
12: return v⃗
```

More Knobs

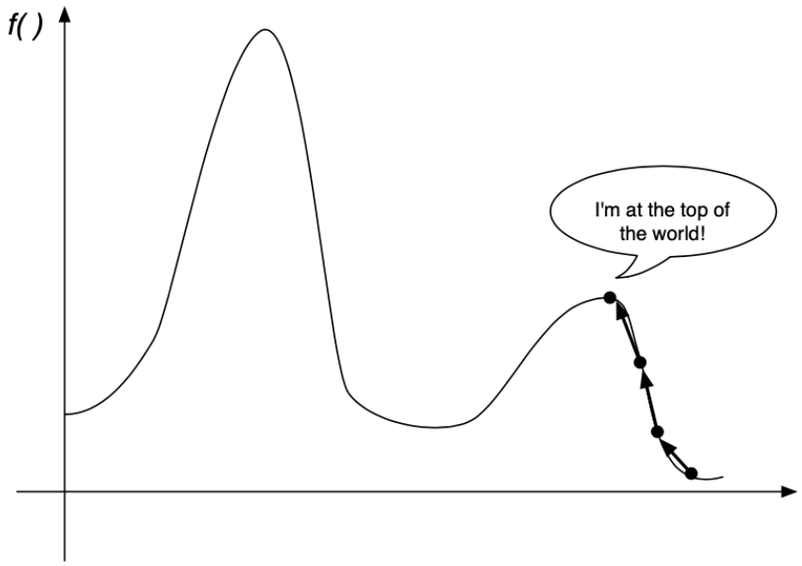
Notice that Alg 8 contains two new hyper-parameters: p and r

We'll assume *min* and *max* are natural bounds determined by the problem

Spinal Tap, 1984

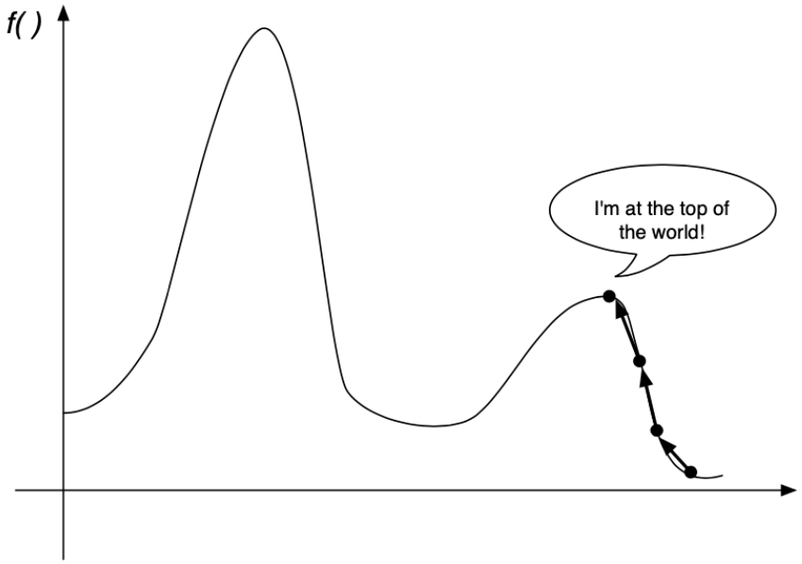
Exploration vs Exploitation

- p and (especially) r determine "step size"
  - small step size: (***exploitation***)
    - "tiptoe" to the top of the hill
    - converges nicely
    - less likely to make "jump" from a local maximum
  - large step size: (***exploration***)
    - "leaps and bounds"
    - faster ascent, but more likely to "overshoot"
    - more likely to make jumps to other maxima (hopefully global maximum)
    - harder to converge ($< \epsilon$) on a maximum
- Exploration vs exploitation is a *fundamental dilemma* in optimisation!



Global Optimisation

- Hill-climbing algorithms are referred to as ***local optimisation*** algorithms.



Global optimisation - if we run long enough, we will *eventually* find a solution.

- Requires *at the limit* every location in search space is visited.
- Cannot guarantee with a bounded "tweak"

Example - Random Search

Simplest *global optimisation* algorithm…

Algorithm 9 *Random Search*

```
Best ← some initial random candidate solution
repeat
    S ← a random candidate solution
    if Quality(S) > Quality(Best) then
        Best ← S
until Best is the ideal solution or we have run out of time
return Best
```

- Extreme case of *exploration* - find a good solution in a "good" part of the space, don't capitalise on it or *exploit* it
- Contrast with hill climb - extreme case of *exploitation*

Examples of Global Search Algorithms

- Adjust modification procedure
  - tweak occasionally makes large random changes
  - controlled by hyper-parameter that sets balance between E & E
- Adjust selection procedure
  - allow it to go down hills sometimes
- "Hyperjumps"/restarts
  - try a new location occasionally (similar to random tweaks)

…

Examples of Global Search Algorithms

- Use a large(r) sample
  - pick best to exploit, then second best, or sample again?
  - how should you sample? random? grid?
  - how far can you go before sampling becomes more expensive than searching?

What is the *best* thing to do?

Next…

- How can we best combine both exploitation and exploration…?

But before we go…

Is there a *best* choice?

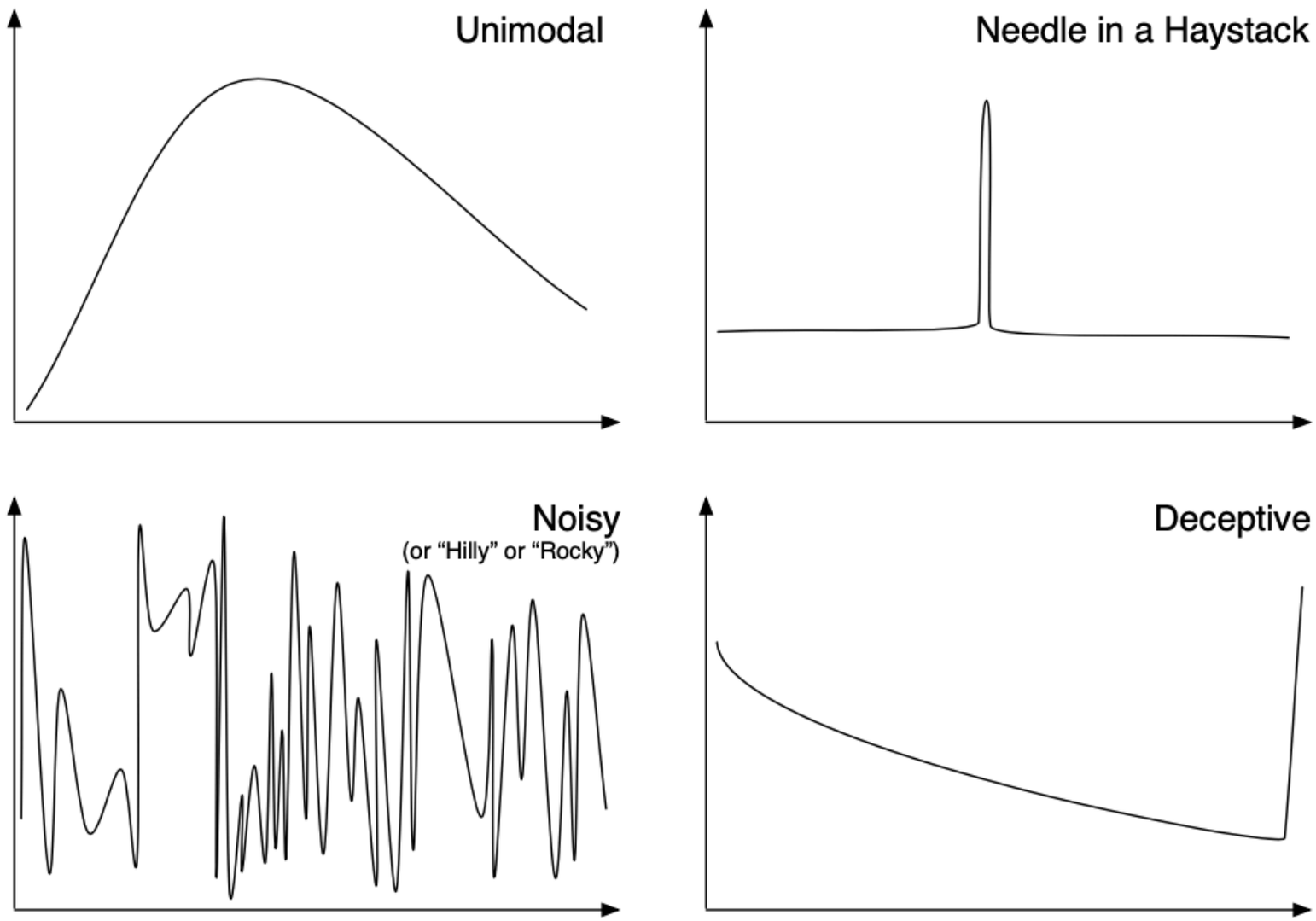
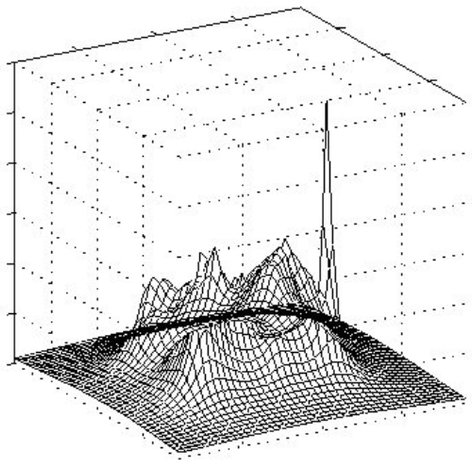


Figure 6 Four example quality functions.

"Foxhole"
eg. Shekel function



There ain't no such thing as a free lunch!

From "No Free Lunch Theorems for Optimization", Wolpert and Macready, IEEE Trans Evolutionary Computation, 1997

"*One might expect that there are pairs of search algorithms A and B such that A performs better than B on average, even if sometimes B outperforms A.*

As an example, one might expect that hill climbing usually outperforms hill descending if one's goal is to find a maximum of the cost function. One might also expect it would outperform a random search in such a context.

One of the main results of this paper is that such expectations are incorrect. We prove two "no free lunch" (NFL) theorems in Section III that demonstrate this and more generally illuminate the connection between algorithms and problems.

Roughly speaking, we show that for both static and time-dependent optimization problems, the average performance of any pair of algorithms across all possible problems is identical."

But as Malcom Fraser didn't say…